Публичная выборка: Lua тестовые задания

MWS Octapi

Exported on 2026-03-31 20:17:03

Table of Contents

1 Ограничения Lua в LowCode....4
1.1 Переменные и типы данных....4
1.2 Конструкции....4
2 Типовые запросы....5
2.1 1. Последний элемент массива....5
2.1.1 Запрос пользователя....5
2.1.2 Ожидаемый ответ....5
2.2 2. Счетчик попыток....5
2.2.1 Запрос пользователя....5
2.2.2 Ожидаемый ответ....5
2.3 3. Очистки значений в переменных....6
2.3.1 Запрос пользователя....6
2.3.2 Ожидаемый ответ....6
2.4 4. Приведение времени к стандарту ISO 8601....7
2.4.1 Запрос пользователя....7
2.4.2 Ожидаемый ответ....9
2.5 5. Проверка типа данных....9
2.5.1 Запрос пользователя....9
2.5.2 Ожидаемый ответ....10
2.6 6. Фильтрация элементов массива 1....11
2.6.1 Запрос пользователя....11
2.6.2 Ожидаемый ответ....11
2.7 7. Дополнение существующего кода....12
2.7.1 Запрос пользователя....12
2.7.2 Ожидаемый ответ....12
2.8 8. Конвертация времени....12
2.8.1 Запрос пользователя....12
2.8.2 Ожидаемый ответ....13

- Ограничения Lua в LowCode
  - Переменные и типы данных
  - Конструкции
- Типовые запросы
  - 1. Последний элемент массива
    - Запрос пользователя
    - Ожидаемый ответ
  - 2. Счетчик попыток
    - Запрос пользователя
    - Ожидаемый ответ
  - 3. Очистки значений в переменных
    - Запрос пользователя
    - Ожидаемый ответ
  - 4. Приведение времени к стандарту ISO 8601
    - Запрос пользователя
    - Ожидаемый ответ
  - 5. Проверка типа данных
    - Запрос пользователя
    - Ожидаемый ответ
  - 6. Фильтрация элементов массива 1
    - Запрос пользователя
    - Ожидаемый ответ
  - 7. Дополнение существующего кода
    - Запрос пользователя
    - Ожидаемый ответ
  - 8. Конвертация времени
    - Запрос пользователя
    - Ожидаемый ответ

1 Ограничения Lua в LowCode

1. Используется **Lua 5.5**
2. Скрипт описывается в формате JsonString **lua{-- действие }lua**
3. **Важно!** В lua скрипте **нельзя** обращаться к переменным с помощью JsonPath, вместо этого необходимо указывать прямое обращение к данным.
4. **Все объявленные в LowCode переменные хранятся в wf.vars.**
5. Переменные, которые получает схема при запуске (из variables) хранятся по пути **wf.initVariables**

1.1 Переменные и типы данных

Разрешено использовать базовые типы данных, такие как:

- **nil** — используется для обозначения отсутствия значения.
- **boolean** — значения true и false.
- **number** — числа (целые и с плавающей запятой).
- **string** — строки
- **array** - массивы. Для создания нового массива в процессе работы скрипта или для объявления существующей переменной массивом используются функции:
  _utils.array.new() - для создания нового массива
  _utils.array.markAsArray(arr) - для объявления существующей переменной массивом
- **table** — таблицы, которые в Lua используются для создания массивов, списков, ассоциативных массивов и объектов.
- **function** — функции

1.2 Конструкции

Разрешено использовать следующие базовые конструкции:

- `if...then...else`
- `while...do...end`
- `for...do...end`
- `repeat...until`

2 Типовые запросы

2.1 1. Последний элемент массива

2.1.1 Запрос пользователя

Из полученного списка email получи последний.

```
{
  "wf": {
    "vars": {
      "emails": [
        "user1@example.com",
        "user2@example.com",
        "user3@example.com"
      ]
    }
  }
}
```

Code block 1 Пример контекста

2.1.2 Ожидаемый ответ

```
return wf.vars.emails[#wf.vars.emails]
```

Code block 2 lua

```
{"lastEmail":"lua{return wf.vars.emails[#wf.vars.emails]}lua"}
```

Code block 3 JSON

2.2 2. Счетчик попыток

2.2.1 Запрос пользователя

Увеличивай значение переменной **try_count_n** на каждой итерации

```
{
  "wf": {
    "vars": {
      "try_count_n": 3
    }
  }
}
```

Code block 4 Пример контекста

2.2.2 Ожидаемый ответ

```
return wf.vars.try_count_n + 1
```

Code block 5 lua

```
{"try_count_n":"lua{return wf.vars.try_count_n + 1}lua"}
```

Code block 6 JSON

2.3 3. Очистки значений в переменных

2.3.1 Запрос пользователя

Для полученных данных из предыдущего REST запроса очисти значения переменных **ID**, **ENTITY_ID**, **CALL**

```
{
  "wf": {
    "vars": {
      "RESTbody": {
        "result": [
          {
            "ID": 123,
            "ENTITY_ID": 456,
            "CALL": "example_call_1",
            "OTHER_KEY_1": "value1",
            "OTHER_KEY_2": "value2"
          },
          {
            "ID": 789,
            "ENTITY_ID": 101,
            "CALL": "example_call_2",
            "EXTRA_KEY_1": "value3",
            "EXTRA_KEY_2": "value4"
          }
        ]
      }
    }
  }
}
```

Code block 7 Пример контекста

2.3.2 Ожидаемый ответ

```lua
result = wf.vars.RESTbody.result

for _, filteredEntry in pairs(result) do
    for key, value in pairs(filteredEntry) do
        if key ~= "ID" and key ~= "ENTITY_ID" and key ~= "CALL" then
            filteredEntry[key] = nil
        end
    end
end

return result
```

Code block 8 lua

```
{
  "result": "lua{\n\tresult = wf.vars.RESTbody.result\n\tfor _,
filteredEntry in pairs(result) do\n\t\tfor key, value in
```

```
pairs(filteredEntry) do\n\t\t\tif key ~= \"ID\" and key ~= \"ENTITY_ID\"
and key ~= \"CALL\" then\n\t\t\t\tfilteredEntry[key] =
nil\n\t\t\tend\n\t\tend\n\tend\n\treturn result\n}lua"
}
```

2.4 4. Приведение времени к стандарту ISO 8601

2.4.1 Запрос пользователя

Преобразуй время из формата 'YYYYMMDD' и 'HHMMSS' в строку в формате ISO 8601 с использованием Lua. Пример в

```
{
  "wf": {
    "vars": {
      "json": {
        "IDOC": {
          "ZCDF_HEAD": {
            "DELIVERY": "123456789",
            "SUBJECT": "Order Confirmation",
            "DATUM": "20231015",
            "TIME": "153000",
            "STATUS": "Confirmed",
            "ROUTE": "Route 66",
            "ROUTE_TXT": "Main Distribution Route",
            "ZCDF_PACKAGES": [
              {
                "id": "PKG001",
                "number": "EXIDV001",
                "weight": "10",
                "volume": "50",
                "length": "100",
                "width": "50",
                "height": "30",
                "items": [
                  {
                    "sku": "SKU001",
                    "externalId": "MAT001",
                    "quantity": "5",
                    "weight": "2"
                  },
                  {
                    "sku": "SKU002",
                    "externalId": "MAT002",
                    "quantity": "3",
                    "weight": "1"
                  }
                ]
              },
              {
                "id": "PKG002",
                "number": "EXIDV002",
                "weight": "20",
                "volume": "60",
                "length": "120",
                "width": "60",
                "height": "40",
                "items": [
                  {
                    "sku": "SKU003",
                    "externalId": "MAT003",
                    "quantity": "10",
                    "weight": "2"
                  }
                ]
              }
            ]
          }
        }
      }
    }
  }
}
```


Code block 9 Пример контекста

2.4.2 Ожидаемый ответ

```
DATUM = wf.vars.json.IDOC.ZCDF_HEAD.DATUM
TIME = wf.vars.json.IDOC.ZCDF_HEAD.TIME

local function safe_sub(str, start, finish)
    local s = string.sub(str, start, math.min(finish, #str))
    return s ~= "" and s or "00"
end

year = safe_sub(DATUM, 1, 4)
month = safe_sub(DATUM, 5, 6)
day = safe_sub(DATUM, 7, 8)
hour = safe_sub(TIME, 1, 2)
minute = safe_sub(TIME, 3, 4)
second = safe_sub(TIME, 5, 6)

iso_date = string.format(
    '%s-%s-%sT%s:%s:%s.00000Z',
    year, month, day,
    hour, minute, second
)

return iso_date
```

Code block 10 lua


```
{"time": "lua{\r\nDATUM = wf.vars.json.IDOC.ZCDF_HEAD.DATUM\r\nTIME =
wf.vars.json.IDOC.ZCDF_HEAD.TIME\r\nlocal function safe_sub(str, start,
finish)\r\n\tlocal s = string.sub(str, start, math.min(finish,
#str))\r\n\treturn s ~= \"\" and s or \"00\"\r\nend \r\nyear =
safe_sub(DATUM, 1, 4)\r\nmonth = safe_sub(DATUM, 5, 6)\r\nday =
safe_sub(DATUM, 7, 8) \r\nhour = safe_sub(TIME, 1, 2)\r\nminute =
safe_sub(TIME, 3, 4)\r\nsecond = safe_sub(TIME, 5, 6) \r\niso_date =
string.format(\r\n\t'%s-%s-%sT%s:%s:%s.00000Z',\r\n\tyear, month,
day,\r\n\thour, minute, second\r\n) \r\nreturn iso_date \r\n}lua"}
```


Code block 11 JSON

2.5 5. Проверка типа данных

2.5.1 Запрос пользователя

Как преобразовать структуру данных так, чтобы все элементы items в **ZCDF_PACKAGES** всегда были представлены в виде массивов, даже если они изначально не являются массивами

```
{
  "wf": {
    "vars": {
      "json": {
        "IDOC": {
          "ZCDF_HEAD": {
            "ZCDF_PACKAGES": [
              { "items": [{"sku": "A"}, {"sku": "B"}] },
              { "items": {"sku": "C"} }
            ]
          }
        }
      }
    }
  }
}
```

Code block 12 Пример контекста

2.5.2 Ожидаемый ответ

```
function ensureArray(t)
    if type(t) ~= "table" then
        return {t}
    end
    local isArray = true
    for k, v in pairs(t) do
        if type(k) ~= "number" or math.floor(k) ~= k then
            isArray = false
            break
        end
    end
    return isArray and t or {t}
end

function ensureAllItemsAreArrays(objectsArray)
    if type(objectsArray) ~= "table" then
        return objectsArray
    end
    for _, obj in ipairs(objectsArray) do
        if type(obj) == "table" and obj.items then
            obj.items = ensureArray(obj.items)
        end
    end
    return objectsArray
end

return ensureAllItemsAreArrays(wf.vars.json.IDOC.ZCDF_HEAD.ZCDF_PACKAGES)
```

Code block 13 lua


```
{"packages": "lua{function ensureArray(t)\r\nif type(t) ~= \"table\"
then\r\nreturn {t}\r\nend\r\n local isArray = true\r\nfor k, v in pairs(t)
do\r\n if type(k) ~= \"number\" or math.floor(k) ~= k then\r\n isArray =
false\r\n   break\r\n  end\r\n end\r\n return isArray and t or {t}\r\n
end\r\n function ensureAllItemsAreArrays(objectsArray)\r\n if
type(objectsArray) ~= \"table\" then\r\n return objectsArray\r\n end\r\n
for _, obj in ipairs(objectsArray) do\r\n if type(obj) == \"table\" and
obj.items then\r\n      obj.items = ensureArray(obj.items)\r\n    end\r\n
```

```
end\r\n return objectsArray\r\n end return
ensureAllItemsAreArrays(wf.vars.json.IDOC.ZCDF_HEAD.ZCDF_PACKAGES)}lua"}}
```

Code block 14 JSON

2.6 6. Фильтрация элементов массива 1

2.6.1 Запрос пользователя

Отфильтруй элементы из массива, чтобы включить только те, у которых есть значения в полях Discount или Markdown.

```
{
  "wf": {
    "vars": {
      "parsedCsv": [
        { "SKU": "A001", "Discount": "10%", "Markdown": "" },
        { "SKU": "A002", "Discount": "", "Markdown": "5%" },
        { "SKU": "A003", "Discount": null, "Markdown": null },
        { "SKU": "A004", "Discount": "", "Markdown": "" }
      ]
    }
  }
}
```

Code block 15 Пример контекста

2.6.2 Ожидаемый ответ

```
local result = _utils.array.new()
local items = wf.vars.parsedCsv

for _, item in ipairs(items) do
    if (item.Discount ~= "" and item.Discount ~= nil) or (item.Markdown ~=
"" and item.Markdown ~= nil) then
        table.insert(result, item)
    end
end

return result
```

Code block 16 lua

```
{
  "result": "lua{\nlocal result = _utils.array.new()\nlocal items =
wf.vars.parsedCsv\n\nfor _, item in ipairs(items) do\n    if
(item.Discount ~= \"\" and item.Discount ~= nil) or (item.Markdown ~= \"\"
and item.Markdown ~= nil) then\n        table.insert(result, item)\n
end\nend\n\nreturn result\n}lua"
}
```

Code block 17 JSON

2.7 7. Дополнение существующего кода

2.7.1 Запрос пользователя

Добавь переменную с квадратом числа

2.7.2 Ожидаемый ответ

```
{"num": "lua{return tonumber('5')}lua", "squared": "lua{local n =
tonumber('5')\\nreturn n * n}lua"}"""
```

2.8 8. Конвертация времени

2.8.1 Запрос пользователя

Конвертируй время в переменной **recallTime** в unix-формат.

```
{
  "wf": {
    "initVariables": {
      "recallTime": "2023-10-15T15:30:00+00:00"
    }
  }
}
```

Code block 18 Пример контекста

2.8.2 Ожидаемый ответ

```
local iso_time = wf.initVariables.recallTime
local days_in_month = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}

if not iso_time or not iso_time:match("^%d%d%d%d%-%d%d%-%d%dT") then
    return nil
end

local function is_leap_year(year)
    return (year % 4 == 0 and year % 100 ~= 0) or (year % 400 == 0)
end

local function days_since_epoch(year, month, day)
    local days = 0
    for y = 1970, year - 1 do
        days = days + (is_leap_year(y) and 366 or 365)
    end
    for m = 1, month - 1 do
        days = days + days_in_month[m]
        if m == 2 and is_leap_year(year) then
            days = days + 1
        end
    end
    days = days + (day - 1)
    return days
end

local function parse_iso8601_to_epoch(iso_str)
    if not iso_str then
        error("Дата не задана (nil)")
    end

    -- Пробуем с миллисекундами
    local year, month, day, hour, min, sec, ms, offset_sign, offset_hour,
offset_min =

iso_str:match("(%d+)-(%d+)-(%d+)T(%d+):(%d+):(%d+)%.(%d+)([+-])(%d+):(%d+)
")

    -- Если не удалось — пробуем без миллисекунд
    if not year then
        year, month, day, hour, min, sec, offset_sign, offset_hour,
offset_min =

iso_str:match("(%d+)-(%d+)-(%d+)T(%d+):(%d+):(%d+)([+-])(%d+):(%d+)")
        ms = 0
    end

    if not year then
        error("Невозможно разобрать дату: " .. tostring(iso_str))
    end

    year = tonumber(year)
    month = tonumber(month)
    day = tonumber(day)
    hour = tonumber(hour)
    min = tonumber(min)
    sec = tonumber(sec)
    ms = tonumber(ms) or 0
    offset_hour = tonumber(offset_hour)
    offset_min = tonumber(offset_min)

    local total_days    = days_since_epoch(year, month, day)
    local total_seconds = total_days * 86400 + hour * 3600 + min * 60 +
sec

    local offset_seconds = offset_hour * 3600 + offset_min * 60
```

```
    if offset_sign == "-" then
        offset_seconds = -offset_seconds
    end

    return total_seconds - offset_seconds
end

local epoch_seconds = parse_iso8601_to_epoch(iso_time)
return epoch_seconds
```


```
{"unix_time":"lua{\n\tlocal iso_time =
wf.initVariables.recallTime\n\tlocal days_in_month = {31, 28, 31, 30, 31,
30, 31, 31, 30, 31, 30, 31}\n\tif not iso_time or not
iso_time:match(\"^%d%d%d%d%-%d%d%-%d%dT\") then\n\t\treturn
nil\n\tend\n\tlocal function is_leap_year(year)\n\t\treturn (year % 4 == 0
and year % 100 ~= 0) or (year % 400 == 0)\n\tend\n\tlocal function
days_since_epoch(year, month, day)\n\t\tlocal days = 0\n\t\tfor y = 1970,
year - 1 do\n\t\t\tdays = days + (is_leap_year(y) and 366 or
365)\n\t\tend\n\t\tfor m = 1, month - 1 do\n\t\t\tdays = days +
days_in_month[m]\n\t\t\tif m == 2 and is_leap_year(year)
then\n\t\t\t\tdays = days + 1\n\t\t\tend\n\t\tend\n\t\tdays = days + (day
- 1)\n\t\treturn days\n\tend\n\tlocal function
parse_iso8601_to_epoch(iso_str)\n\t\tif not iso_str then error(\"Дата не
задана (nil)\") end\n\t\tlocal year, month, day, hour, min, sec, ms,
offset_sign, offset_hour, offset_min
=\n\t\t\tiso_str:match(\"(%d+)-(%d+)-(%d+)T(%d+):(%d+):(%d+)%.(%d+)([+-])(
%d+):(%d+)\")\n\t\tif not year then\n\t\t\tyear, month, day, hour, min,
sec, offset_sign, offset_hour, offset_min
=\n\t\t\t\tiso_str:match(\"(%d+)-(%d+)-(%d+)T(%d+):(%d+):(%d+)([+-])(%d+):
(%d+)\")\n\t\t\tms = 0\n\t\tend\n\t\tif not year then error(\"Невозможно
разобрать дату: \" .. tostring(iso_str)) end\n\t\tyear = tonumber(year);
month = tonumber(month); day = tonumber(day)\n\t\thour = tonumber(hour);
min = tonumber(min); sec = tonumber(sec)\n\t\tms = tonumber(ms) or
0\n\t\toffset_hour = tonumber(offset_hour); offset_min =
tonumber(offset_min)\n\t\tlocal total_days = days_since_epoch(year, month,
day)\n\t\tlocal total_seconds = total_days * 86400 + hour * 3600 + min *
60 + sec\n\t\tlocal offset_seconds = offset_hour * 3600 + offset_min *
60\n\t\tif offset_sign == \"-\" then offset_seconds = -offset_seconds
end\n\t\treturn total_seconds - offset_seconds\n\tend\n\tlocal
epoch_seconds = parse_iso8601_to_epoch(iso_time)\n\treturn
epoch_seconds\n}lua"}
```


Code block 19 lua